

27 | Rust Bevy游戏开发：用300行代码做一个贪吃蛇游戏

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天我们一起学习 Rust 游戏编程技术。这节课我们会基于 Bevy 游戏框架来开发一个入门版的贪吃蛇游戏。

Rust 生态内目前已经有不少很不错的游戏开发框架，而 Bevy 是最热门的那一个，目前（2023 年 12 月）最新版本是 0.12，还处在积极开发的过程中。Bevy 框架和 Axum Web 框架、Slint 框架给人的感觉有点儿像，都很简单、优美、灵活。用 Bevy 框架写游戏非常惬意，已经有不少人在尝试使用 Bevy 开发自己的独立游戏，目前有三款（Molecoole、Tiny Glade、Roids）已经上架或即将上架 Steam。

用 Bevy 开发的游戏能够运行在 Windows、macOS、Linux, Web 浏览器等平台。

Bevy 框架

Bevy 框架是一个数据驱动的游戏开发框架（引擎），其核心是一个 ECS。

ECS

ECS 是 Entity Component System 的缩写，意思是实体 - 组件 - 系统。它是一种编程范式，这种范式非常有趣，也非常有潜力，现在的主流游戏引擎都开始支持这种编程范式了。这种范式是与传统的 OOP（面向对象编程）范式相对的，跟 Rust 的 trait 的设计理念有一些相似之处。

我们用一个例子来说明 ECS 是怎样对问题进行建模的。假如现在有这样一幅画面：一个下午，在温暖的家里面，爸爸 D 正在边吃甜点边看书，妈妈 M 在边吃甜点边玩手机，儿子 S 在和狗狗 B 玩。你想想，这个场景如果用 OOP 方式，应该如何建模呢？而用 ECS 范式可以这样建立模型：

Entity	组件：Role	组件：Name	组件：Snack	组件：Book	组件：Phone	组件：Playmate
1	Dad	D	✓	✓		
2	Mommy	M	✓		✓	
3	Son	S				4
4	Dog	B				3

 极客时间

Systems:

 复制代码

```
1 system1: dad_task(query: Query<>)
2 system2: mom_task(query: Query<>)
3 system3: son_task(query: Query<>)
4 system4: dog_task(query: Query<>)
```

这样这个模型就建立好了。

我们用类似数据库 table 或者 Excel 的 datasheet 的形式来描述 Entity 与 Component 之间的关系。Entity 就用简单的数字来表示，只要能区分不同的实体就可以。然后我们定义了 Role、Name、Snack、Book、Phone、Playmat 6 种 Component。

这些 Components 就像数据库 table 的列。但是与数据库不同的是，在 ECS 范式中，这个 table 的列是可以随着程序的运行而动态增加、减少的。另外一个重要的不同是，并不是所有的 Entity 都强制拥有所有的 Component（列），每个 Entity 其实只关心自己需要的 Components 就行了。因此，这里的 table 表示在数据上的话，更像一个稀疏矩阵或集合。

这其实体现了一种设计哲学：**将所有信息铺平，用组合的方式来建模**。是不是与 Rust 的 trait 设计哲学有相似性？

你可以把组件 Component 看作一组属性的集合，将属性按 Component 拆开来放置有利于复用。在这个例子里，4 个实体都复用 Role 组件和 Name 组件，Dad 和 Mommy 复用 Snack 组件，Son 和 Dog 复用 Playmate 组件。

而 System 就是行为或逻辑，用来处理上面建好的表格数据。一个 System 对应 Bevy 中，就是普通的 Rust 函数。当然，这个函数首先要有办法拿到上述表格（世界状态）的操作权力才行，操作的方法就是 Query 检索。

关于 ECS 与 OOP 的对比，你可以参考 [🔗 这里](#)。

资源 (Resource)

对于在整个系统中，只会存在一份的，可以把它定义为 Resource。比如外部的图片素材、模型文件、音频素材等。另外还包含定时器实例、设备抽象等。你可以将资源想象成编程范式中的 Singleton（单例）。

事件 (Event)

游戏世界中，有无处不在的并行任务，比如 10 辆坦克同时寻路前进，任务之间的通信，最好是通过事件来沟通。这是一种用于解耦逻辑的编程范式。

世界状态

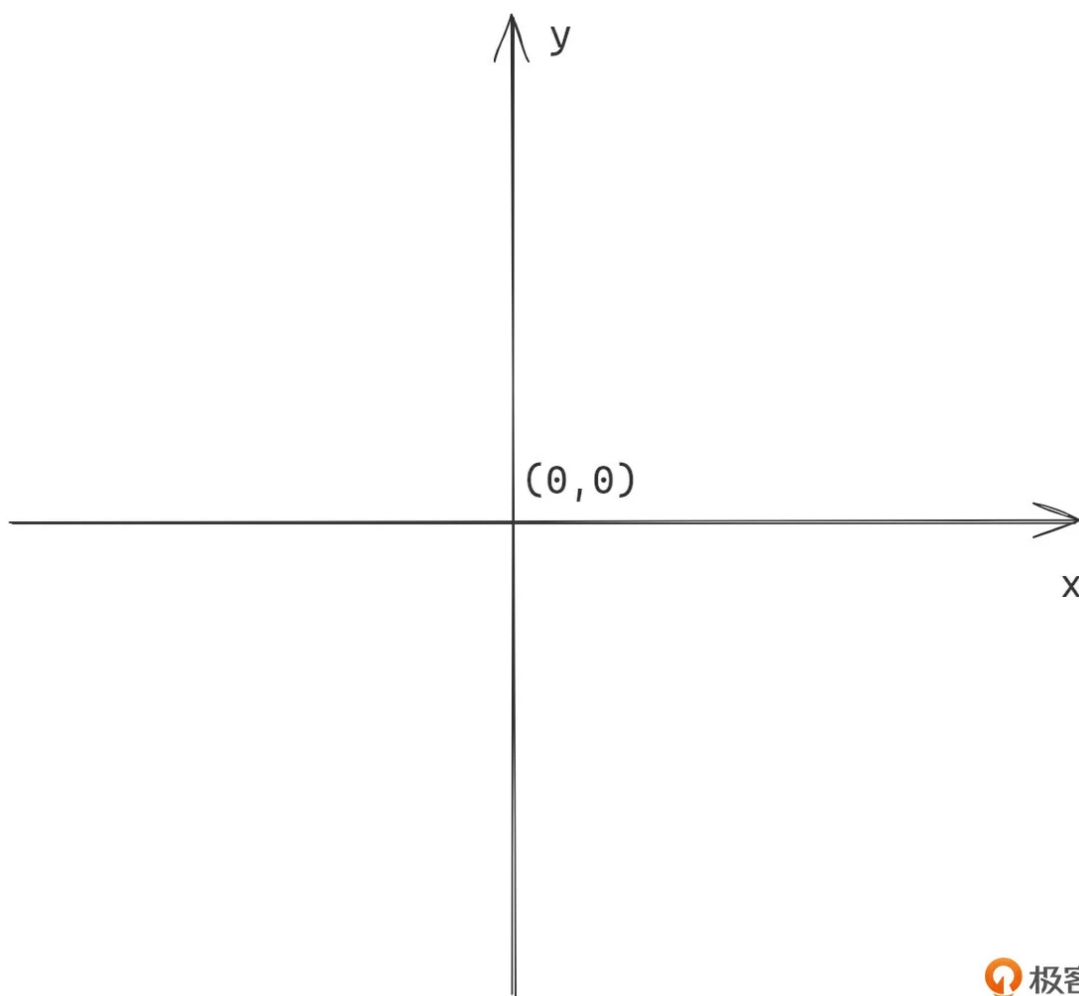
基于 ECS 的设计，那张大表 table 其实就是一个世界状态。基于 ECS 的游戏引擎，就需要在内部维护这样一个世界状态。这个世界状态的维护非常关键，需要用高效的数据结构和算法实现。在 Bevy 中具体用什么数据结构来维护的，你可以参考 [🔗 这里](#)。

固定帧率

游戏一般会以固定帧率运行，比如每秒 60 帧。游戏引擎通常会封装好这个，将你从帧率刷新的任务中释放出来，专注于游戏逻辑的设计。你只需要知道，你写的游戏逻辑会每秒执行 60 次，也就是 60 个滴答 tick。

坐标系

Bevy 的 2D 默认的坐标系是原点在窗口正中间的一个正坐标系，像下面这样：



摄像机 (Camera)

游戏引擎中都会有 Camera 的概念，3D 游戏的画面渲染严重依赖于 Camera。2D 游戏不太关心 Camera，但使用 Camera 也会有放大缩小的效果，默认 Bevy 的 Camera 在坐标系的

Z 轴上，也就是当前你眼睛所处的位置。

性能

借助于 ECS 范式，加上 Rust 强大的无畏并发能力，Bevy 能够让你的 systems 尽可能地运行在多个 CPU 核上，也就是并行运算。所以 Bevy 的基础性能是非常棒的，关于 benchmarks 的讨论，你可以看 [🔗 这里](#)。

有了这些基础知识的铺垫，我们下面进入实战环节吧。

实战贪吃蛇

这里我先给出完整代码的 [🔗 链接](#)，你最好下载下来边运行边对照下面的内容学习。

第 1 步：引入 Bevy 库

很简单，引入 Bevy 库，创建一个 App 实例。

[📄 复制代码](#)

```
1 use bevy::prelude::*;
2
3 fn main() {
4     App::new().run();
5 }
```

这个程序运行后马上就结束了，没有任何输出，也没有窗口打开。

第 2 步：创建窗口

加入默认 Plugin 集合，里面有个主事件循环，还有个创建窗口的功能。然后我们需要设置 2D 的 Camera。

[📄 复制代码](#)

```
1 use bevy::prelude::*;
2
3 fn main() {
```

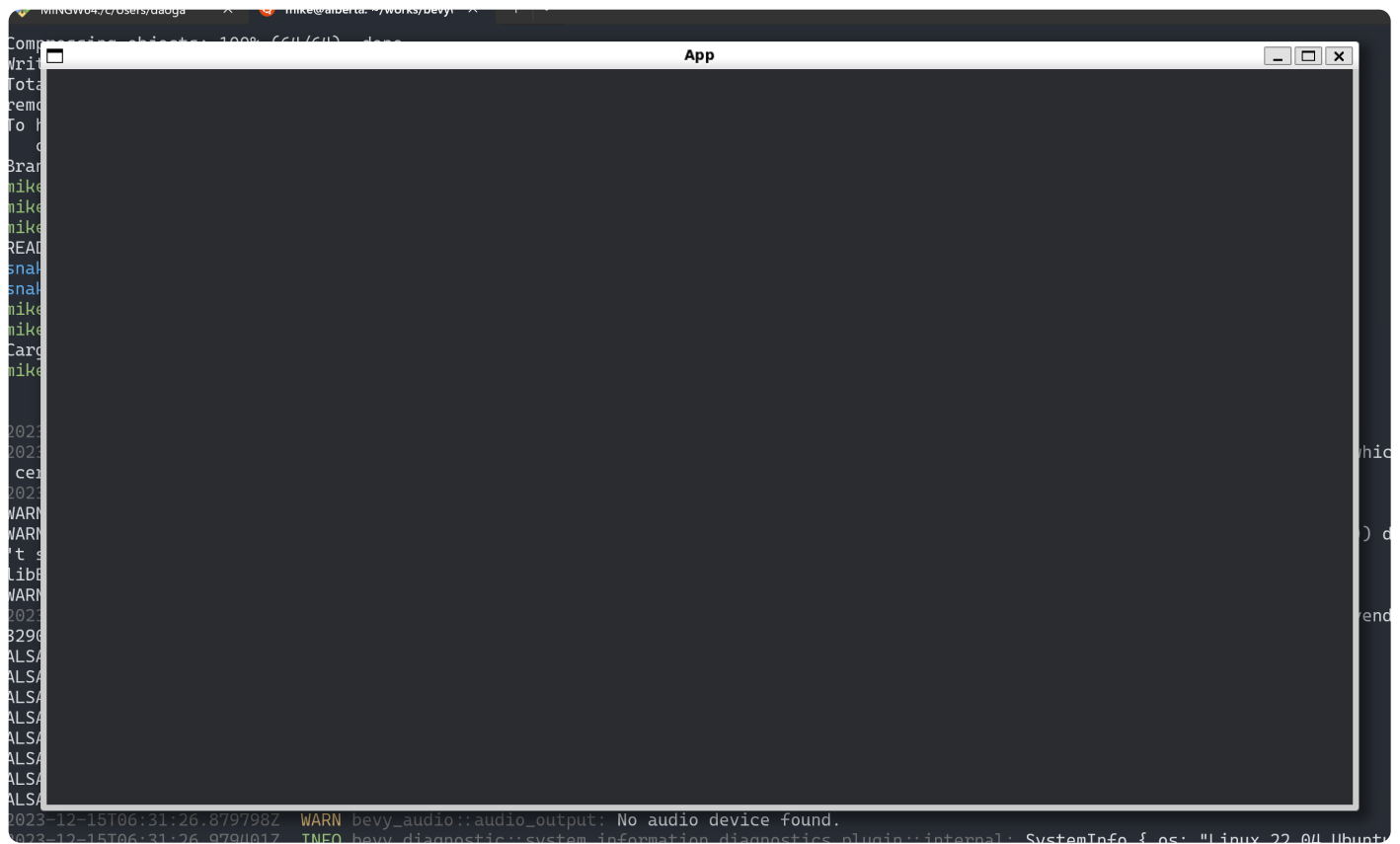
```

4     App::new()
5         .add_plugins(DefaultPlugins)
6         .add_systems(Startup, setup_camera)
7         .run();
8 }
9
10 fn setup_camera(mut commands: Commands) {
11     commands.spawn(Camera2dBundle::default());
12 }

```

由于引擎本身是一个托管系统（带主循环的 Runtime），我们要在这个引擎所维护的世界状态里添加（或删除）新的东西，必须使用 Commands 这种任务指令形式。你可以把它想象成总线或消息队列编程模型。

这一步运行后，弹出一个窗口，并且渲染默认背景色。



第 3 步：画出蛇的头

这一步我们添加一个新函数，创建蛇的头，然后用 add_systems 添加到 bevy runtime 中。你可以看一下代码发生的变化。

```
1 const SNAKE_HEAD_COLOR: Color = Color::rgb(0.7, 0.7, 0.7);
2
3 #[derive(Component)]
4 struct SnakeHead;
5
6 //
7 .add_systems(Startup, (setup_camera, spawn_snake))
8 //
9
10 fn spawn_snake(mut commands: Commands) {
11     commands
12         .spawn(SpriteBundle {
13             sprite: Sprite {
14                 color: SNAKE_HEAD_COLOR,
15                 ..default()
16             },
17             transform: Transform {
18                 scale: Vec3::new(10.0, 10.0, 10.0),
19                 ..default()
20             },
21             ..default()
22         })
23         .insert(SnakeHead);
24 }
```

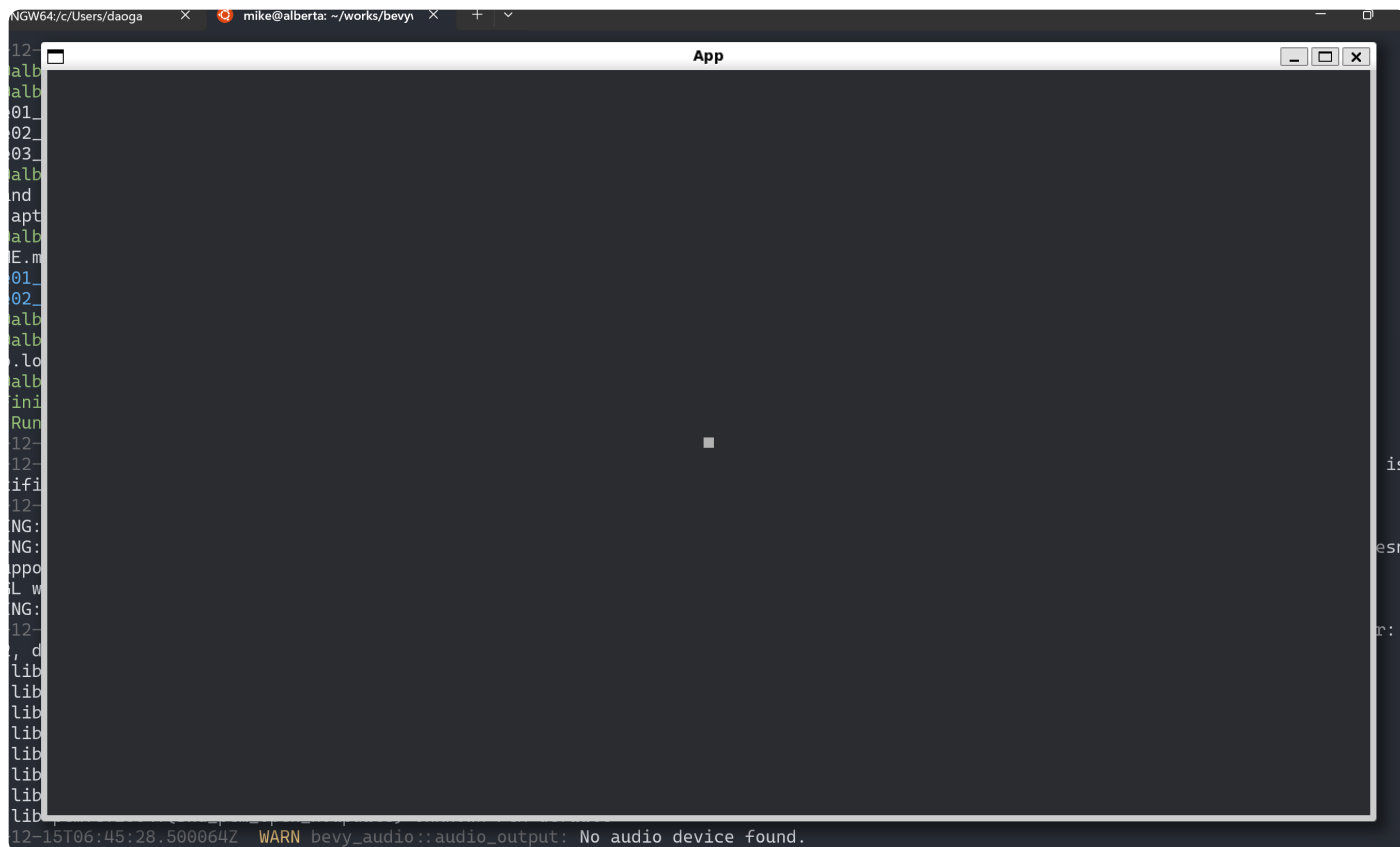
我们用 struct 定义了 SnakeHead Component，它没有任何字段。没关系，目前一个类型名字符号就能表达我们的意思，当一个 tag 用。你继续往后面看。

你可以看到，一个 system 就是一个普通的 Rust 函数。SpriteBundle 是一个 Component Bundle，也就是组件包，可以把一组 components 组合在一起，SpriteBundle 里面就有 Sprite、Transform 等 components。Sprite 就是图片精灵的意思，是游戏里面表达角色的基本方法。Transform 抽象的是角色的运动，有位移、旋转和拉伸变换三种形式。

`spawn_snake()` system 目的就是创建这个蛇的头，它作为一个 entity 被插入到世界状态中。`.insert(SnakeHead)` 把 SnakeHead 这个 Component 添加到这个刚创建好的 entity 上面。

`add_systems()` 中的第一个参数 `Startup`，用来表示这是游戏启动的时候执行的 systems。它们只执行一次，多个 systems 写在元组里面，更简洁。

你可以看一下这一步的运行效果，窗口中间出现了一个小方块，那就是蛇的头。



第 4 步：让这条蛇动起来

这里我给出这一步添加的代码，我们边看边解读。

复制代码

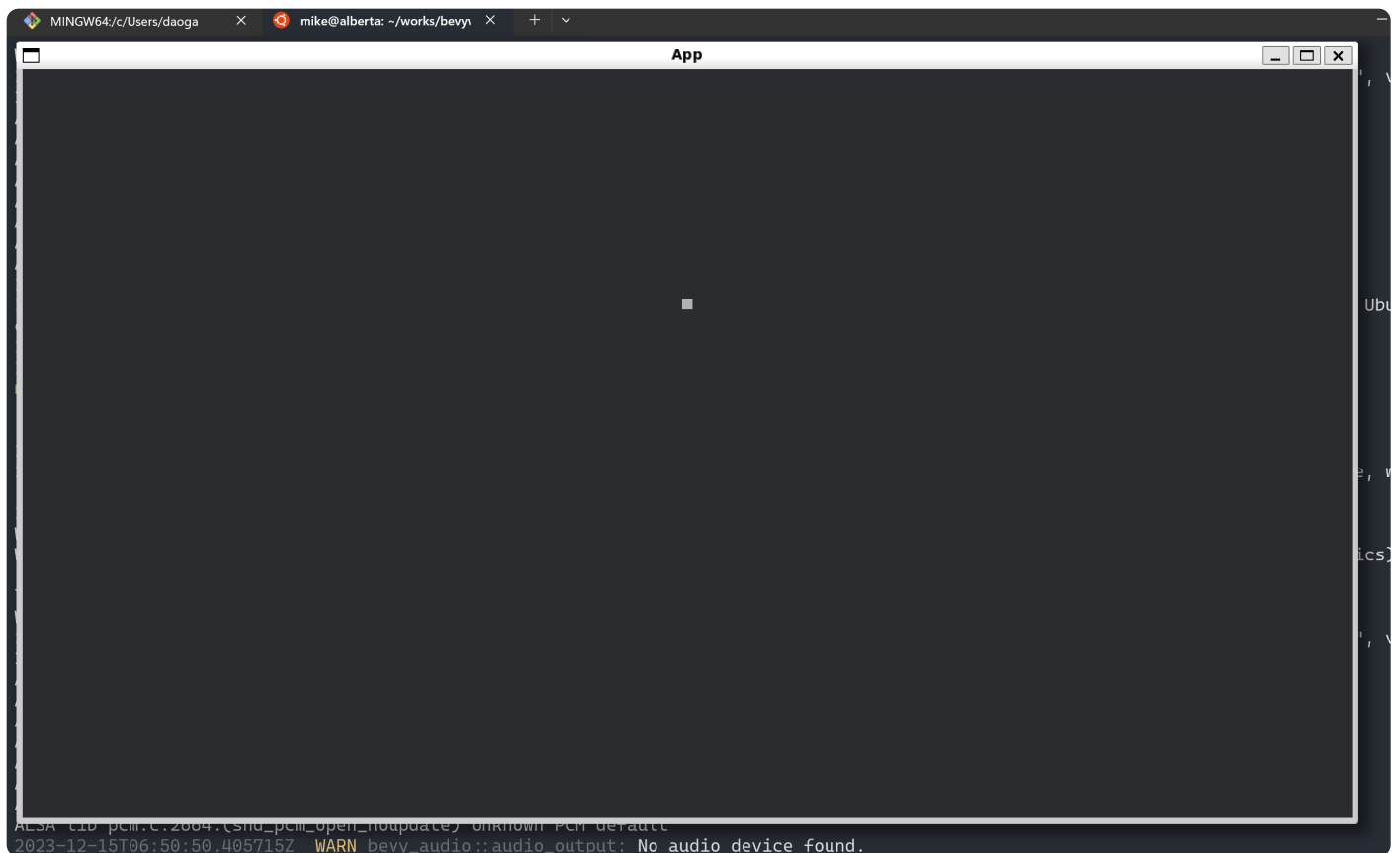
```
1 .add_systems(Update, snake_movement)
2
3 fn snake_movement(mut head_positions: Query<(&SnakeHead, &mut Transform)>()) {
4     for (_head, mut transform) in head_positions.iter_mut() {
5         transform.translation.y += 2.;
6     }
7 }
```

这个 `snake_movement()` 就是处理蛇运动的 system，请注意参数

是 `Query(&SnakeHead, &mut Transform)>` 类型，它的意思是从世界状态中去查找同时拥有 `SnakeHead`、`Transform` 两种 `Components` 的 `entity`，它定义了一个迭代器，并且 `Transform` 的实例还是可以修改的。遍历这个迭代器，其实目前只有一个 `entity`，更新负责管理它运动的 `transform` 实例。 `transform.translation.y += 2.` 就是纵向坐标向上移动 2 个像素。

`add_systems()` 的第一个参数 `Update`，表示这个 `system` 是运行在游戏运行过程中的，每一帧都需要更新一次（执行一次这个 `system`），也就是这个函数 1 秒会执行 60 次。

运行后你会发现这个小方块在自动向上移动，效果如下：



第 5 步：控制这条蛇的方向

下面我们需要控制蛇的方向，上下左右四个方向。这一步就是给 `snake_movement system` 填充内容。

```
1 fn snake_movement(
```

复制代码


```

2     keyboard_input: Res<Input<KeyCode>>,
3     mut head_positions: Query<&mut Transform, With<SnakeHead>>,
4 ) {
5     for mut transform in head_positions.iter_mut() {
6         if keyboard_input.pressed(KeyCode::Left) {
7             transform.translation.x -= 2.;
8         }
9         if keyboard_input.pressed(KeyCode::Right) {
10            transform.translation.x += 2.;
11        }
12        if keyboard_input.pressed(KeyCode::Down) {
13            transform.translation.y -= 2.;
14        }
15        if keyboard_input.pressed(KeyCode::Up) {
16            transform.translation.y += 2.;
17        }
18    }
19 }

```

`Input<KeyCode>` 是 Bevy 系统级的资源，用于表示输入设备，这里是键盘设备。要访问资源就在 `system` 里用 `Res<T>` 这种参数类型。`keyboard_input.pressed()` 用于判断是否键盘按下了。

运行后，你就可以用方向键控制这个小方块的运动方向了。

第 6 步：将窗口网格化

默认 Bevy 的窗口坐标粒度是以屏幕的逻辑像素为单位的。而像贪吃蛇这种游戏，会将整个画布分成一个个正方形的小方格。具体怎么做，你可以看一下这一步变化的代码。

 复制代码

```

1  const ARENA_WIDTH: u32 = 10;
2  const ARENA_HEIGHT: u32 = 10;
3
4  #[derive(Component, Clone, Copy, PartialEq, Eq)]
5  struct Position {
6      x: i32,
7      y: i32,
8  }
9
10 #[derive(Component)]

```

```
11 struct Size {
12     width: f32,
13     height: f32,
14 }
15
16 impl Size {
17     pub fn square(x: f32) -> Self {
18         Self {
19             width: x,
20             height: x,
21         }
22     }
23 }
24
25 //
26 .add_systems(Update, (snake_movement, size_scaling, position_translation)
27
28 //
29 .insert(Position { x: 3, y: 3 })
30 .insert(Size::square(0.8));
31 //
32
33 // 计算方块元素的大小
34 fn size_scaling(primary_query: Query<&Window, With<bevy::window::PrimaryWindow>>,
35     let window = primary_query.get_single().unwrap();
36     for (sprite_size, mut transform) in q.iter_mut() {
37         transform.scale = Vec3::new(
38             sprite_size.width / ARENA_WIDTH as f32 * window.width() as f32,
39             sprite_size.height / ARENA_HEIGHT as f32 * window.height() as f32,
40             1.0,
41         );
42     }
43 }
44
45 // 计算位移
46 fn position_translation(primary_query: Query<&Window, With<bevy::window::PrimaryW
47     fn convert(pos: f32, bound_window: f32, bound_game: f32) -> f32 {
48         let tile_size = bound_window / bound_game;
49         pos / bound_game * bound_window - (bound_window / 2.) + (tile_size / 2.)
50     }
51
52     let window = primary_query.get_single().unwrap();
53     for (pos, mut transform) in q.iter_mut() {
54         transform.translation = Vec3::new(
55             convert(pos.x as f32, window.width() as f32, ARENA_WIDTH as f32),
56             convert(pos.y as f32, window.height() as f32, ARENA_HEIGHT as f32),
57             0.0,
58         );
59     }
```

这一步，我们添加了 Position 和 Size 两种 Components。用来控制蛇头的逻辑位置和显示大小。新增了 `size_scaling` 和 `position_translation` 两个 system，用来在每一帧计算蛇头的尺寸和位置。

具体的算法理解上要注意的就是，坐标的原点是在窗口正中央，转换后的网格 grid 的坐标原点在窗口左下角。

你可以看一下这一步运行后的效果。



你可以看到，蛇的头的大小（为一个网格大小的 0.8）和位置已经变化了。这里的位置在 (3, 3)，网格总大小为 (10, 10)，左下角为 (0, 0)。

第 7 步：让蛇按网格移动

下面要让蛇的运动适配网格。你看一下这一步改动的代码。

```
1 fn snake_movement(  
2     keyboard_input: Res<Input<KeyCode>>,  
3     mut head_positions: Query<&mut Position, With<SnakeHead>>,  
4 ) {  
5     for mut pos in head_positions.iter_mut() {  
6         if keyboard_input.pressed(KeyCode::Left) {  
7             pos.x -= 1;  
8         }  
9         if keyboard_input.pressed(KeyCode::Right) {  
10             pos.x += 1;  
11         }  
12         if keyboard_input.pressed(KeyCode::Down) {  
13             pos.y -= 1;  
14         }  
15         if keyboard_input.pressed(KeyCode::Up) {  
16             pos.y += 1;  
17         }  
18     }  
19 }
```

这一步我们要更新蛇头的逻辑坐标，也就是上一步定义那个 Position component 的实例。现在你可以通过方向键将这个小矩形块移动到其他位置。



第 8 步：配置窗口比例和尺寸

默认打开的窗口是长方形的，我们要给它配置成方形。你可以看一下这一步的变化代码。

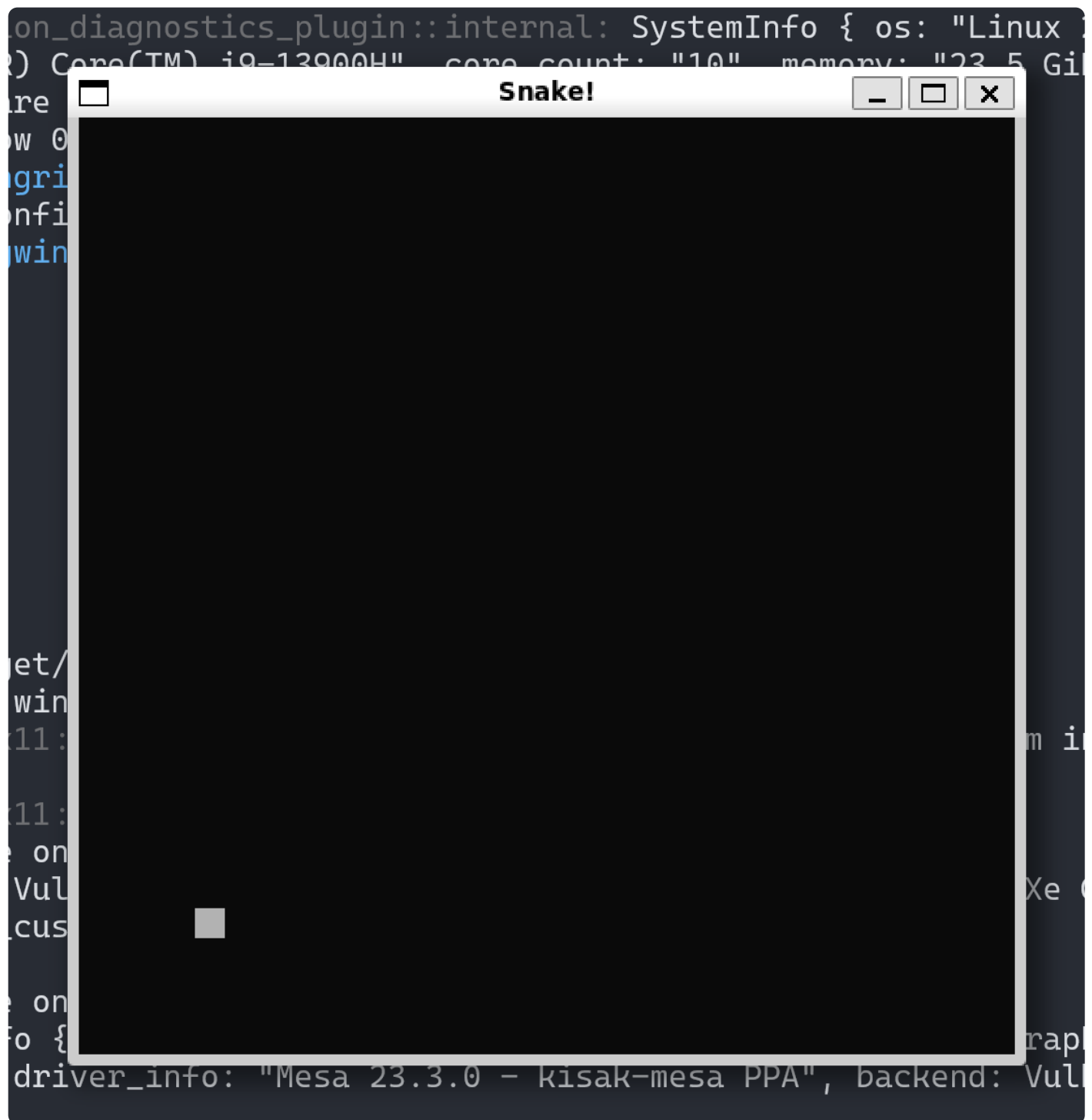
[📄 复制代码](#)

```
1  const ARENA_WIDTH: u32 = 25;
2  const ARENA_HEIGHT: u32 = 25;
3
4      //
5      .add_plugins(DefaultPlugins.set(WindowPlugin {
6          primary_window: Some( Window {
7              title: "Snake!".to_string(),
8              resolution: bevy::window::WindowResolution::new( 500.0, 500.0
9              ..default()
10          }),
11          ..default()
12      })
13  )
14  .insert_resource(ClearColor(Color::rgb(0.04, 0.04, 0.04)))
15  //
```

这一步我们做了 4 件事情。

1. 设置窗口尺寸为 500px x 500px。
2. 设置窗口标题为 Snake!。
3. 设置窗口填充背景颜色为 Color::rgb(0.04, 0.04, 0.04)。
4. 分割窗口 grid 为更大一点的数字，比如 25x25。

你看一下这一步的效果。



离我们期望的样子越来越近了。

第 9 步：随机产生食物

下面要开始产生食物。食物我们也用另一种小方块来表示。你看一下这一步变化的代码。

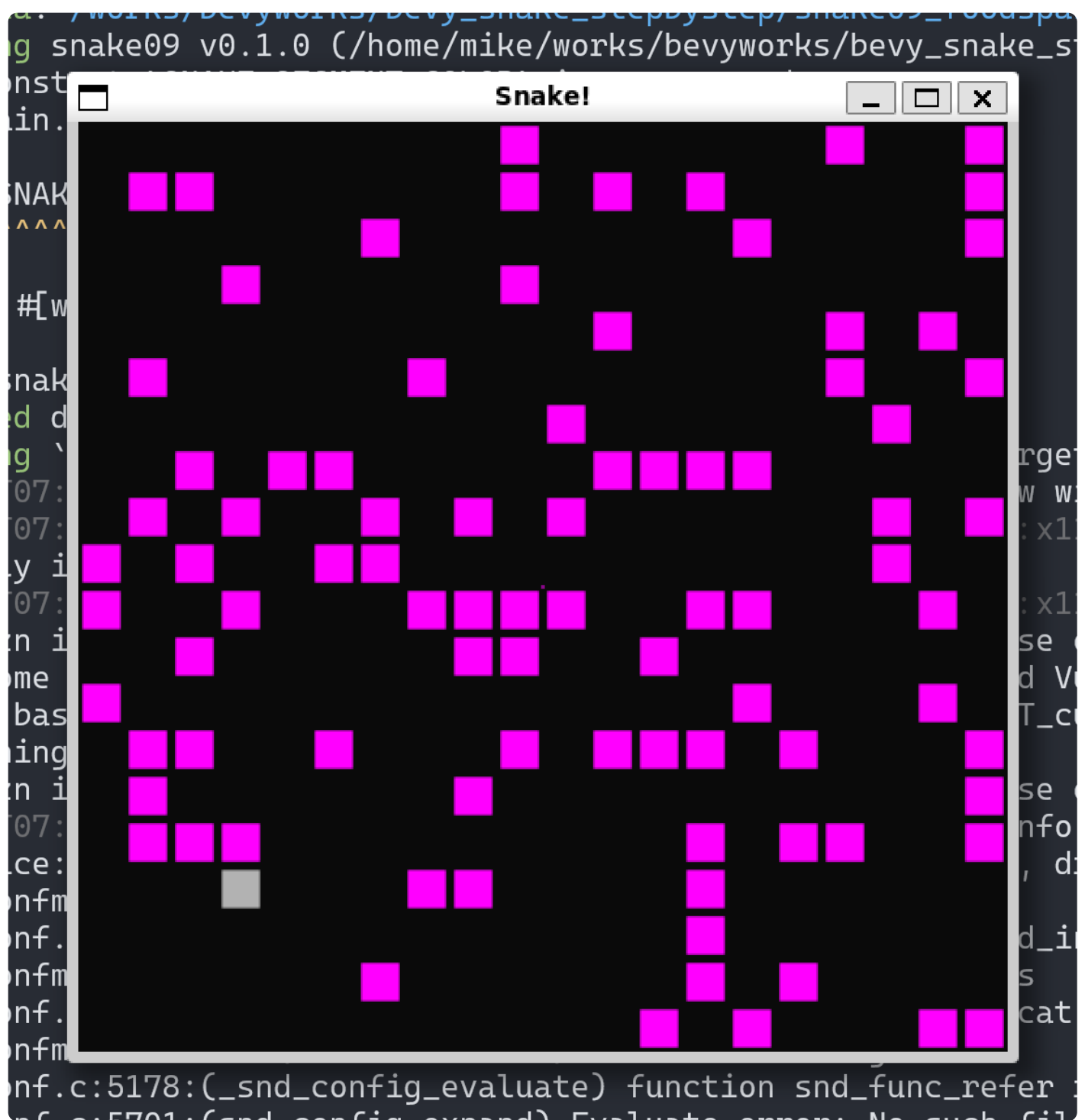
```
1 const FOOD_COLOR: Color = Color::rgb(1.0, 0.0, 1.0);
```

 复制代码

```
2  #[derive(Component)]
3  struct Food;
4
5      .add_systems(Update, food_spawner)
6
7  fn food_spawner(mut commands: Commands) {
8      commands
9          .spawn(SpriteBundle {
10              sprite: Sprite {
11                  color: FOOD_COLOR,
12                  ..default()
13              },
14              ..default()
15          })
16          .insert(Food)
17          .insert(Position {
18              x: (random::<f32>() * ARENA_WIDTH as f32) as i32,
19              y: (random::<f32>() * ARENA_HEIGHT as f32) as i32,
20          })
21          .insert(Size::square(0.8));
22  }
23 }
```

食物随机产生，所以需要用到 random 函数。同样，我们定义了 Food 这个 Component，然后定义了 food_spawner system，并添加到 runtime 中去。创建的食物 entity 上带有 Sprite、Food、Position、Size 等 components。

可以想象，这个创建食物的 system1 秒会执行 60 次，也就是 1 秒钟会创建 60 个食物，速度太快了。



第 10 步：使用定时器产生食物

下面我们要控制食物的产生速度，比如 2 秒产生一颗食物。我们来看这一步变化的代码。

复制代码

```
1 #[derive(Resource)]
2 struct FoodSpawnTimer(Timer);
3
4     .insert_resource(FoodSpawnTimer(Timer::from_seconds(
```



```

5         1.0,
6         TimerMode::Repeating,
7     )))
8
9 fn food_spawner(
10     mut commands: Commands,
11     time: Res<Time>,
12     mut timer: ResMut<FoodSpawnTimer>,
13 ) {
14     // 如果时间未到 2s 就立即返回
15     if !timer.0.tick(time.delta()).finished() {
16         return;
17     }
18     commands
19         .spawn(SpriteBundle {
20             sprite: Sprite {
21                 color: FOOD_COLOR,
22                 ..default()
23             },
24             ..default()
25         })
26         .insert(Food)
27         .insert(Position {
28             x: (random::<f32>() * ARENA_WIDTH as f32) as i32,
29             y: (random::<f32>() * ARENA_HEIGHT as f32) as i32,
30         })
31         .insert(Size::square(0.8));
32 }

```

Timer 类型是 Bevy 提供的定时器类型，我们用 newtype 模式定义一个自己的定时器，它是一种资源（全局唯一）。我们使用 `insert_resource()` 将这个资源初始化并插入到托管系统中去。

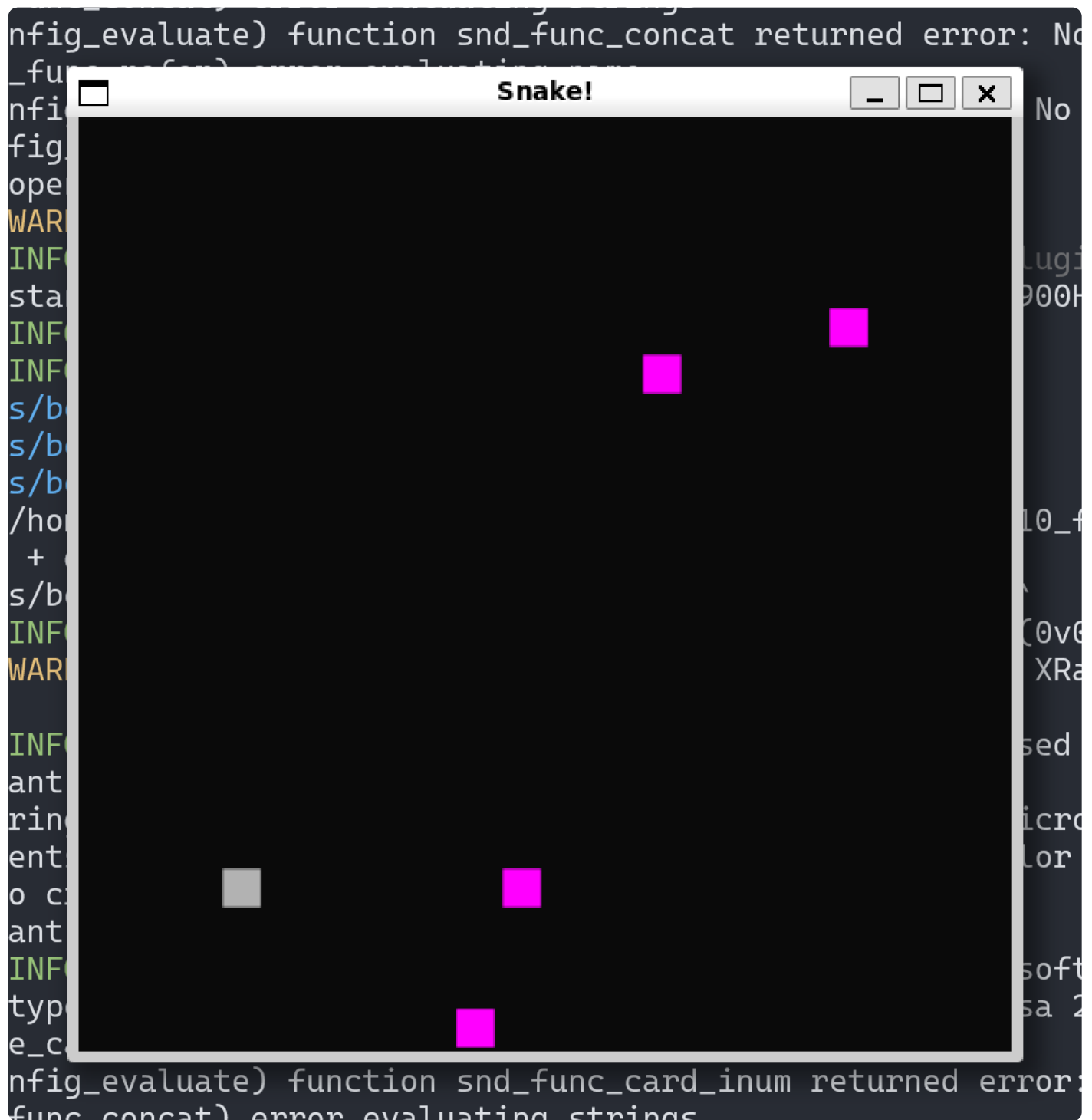
然后在 `food_spawner system` 中使用 `ResMut<FoodSpawnTimer>` 这种形式来使用资源。同时用 `Res<Time>` 这种形式来获取游戏中的时间，这个也是 Bevy 引擎提供的。细心的你可能发现了，Bevy 采用的也是声明式参数实现，和前面课程讲到的 Axum 风格一样。这些参数顺序是可以变的！在这里你可以体会到 Rust 强大的表达能力。

我们再来看一句。

```
1     if !timer.0.tick(time.delta()).finished() {  
2         return;  
3     }
```

这一句表示每次执行这个 `food_spawner system` (1 秒执行 60 次) 时，先判断当前流逝了多少时间，如果定时器的一次间隔还没到，就直接返回，不执行这个函数后面的部分，也就不产生一个食物了。这样就实现了控制食物产生速率的目的。

你可以看一下运行效果。



现在 2 秒产生一颗食物，速度比刚才慢多了。

第 11 步：让蛇自动前进

下面我们要让蛇自己动起来，而且也要控制它的运动速率。同样的我们会用定时器方法。

你来看这一步改动的代码。

```
1 #[derive(Resource)]
2 struct BTimer(Timer);
3
4 #[derive(Component)]
5 struct SnakeHead {
6     direction: Direction,
7 }
8
9 #[derive(PartialEq, Copy, Clone)]
10 enum Direction {
11     Left,
12     Up,
13     Right,
14     Down,
15 }
16
17 impl Direction {
18     fn opposite(self) -> Self {
19         match self {
20             Self::Left => Self::Right,
21             Self::Right => Self::Left,
22             Self::Up => Self::Down,
23             Self::Down => Self::Up,
24         }
25     }
26 }
27
28 // 插入定时器资源
29 .insert_resource(BTimer(Timer::from_seconds(
30     0.20,
31     TimerMode::Repeating,
32 )))
33 // 更新Update模式下的system集
34 .add_systems(Update, (
35     snake_movement_input.before(snake_movement),
36     snake_movement,
37     size_scaling,
38     position_translation))
39 )
40 // 根据用户键盘行为，预处理蛇的前进方向
41 fn snake_movement_input(
42     keyboard_input: Res<Input<KeyCode>>,
43     mut heads: Query<&mut SnakeHead>) {
44     if let Some(mut head) = heads.iter_mut().next() {
45         let dir: Direction = if keyboard_input.just_pressed(KeyCode::Left) {
46             Direction::Left
47         } else if keyboard_input.just_pressed(KeyCode::Down) {
48             Direction::Down
49         }
50     }
51 }
```

```

48     } else if keyboard_input.just_pressed(KeyCode::Up) {
49         Direction::Up
50     } else if keyboard_input.just_pressed(KeyCode::Right) {
51         Direction::Right
52     } else {
53         head.direction
54     };
55     if dir != head.direction.opposite() {
56         head.direction = dir;
57     }
58 }
59 }
60
61 // 蛇的运动system
62 fn snake_movement(
63     mut heads: Query<(&mut Position, &SnakeHead)>,
64     time: Res<Time>,
65     mut timer: ResMut<BTimer>,
66 ) {
67     // 如果时间未到 0.2s 就立即返回
68     if !timer.0.tick(time.delta()).finished() {
69         return;
70     }
71
72     if let Some((mut head_pos, head)) = heads.iter_mut().next() {
73         match &head.direction {
74             Direction::Left => {
75                 head_pos.x -= 1;
76             }
77             Direction::Right => {
78                 head_pos.x += 1;
79             }
80             Direction::Up => {
81                 head_pos.y += 1;
82             }
83             Direction::Down => {
84                 head_pos.y -= 1;
85             }
86         };
87     }
88 }

```

类似地，我们定义了 BTimer 来控制蛇的自动行走，0.2 秒走一格。同时，我们现在可以给蛇指定行走的方向了，因此新定义了 Direction 枚举，并在 SnakeHead Component 里添加了 direction 字段。

代码中的 `snake_movement_input.before(snake_movement)` 表示明确指定 `snake_movement_input` 在 `snake_movement system` 之前处理。因为 bevy 默认会尽可能并行化，所以这样指定能够明确 `system` 的执行顺序，不然可能是乱序执行的。

第 12 步：定义蛇身

下面是定义蛇的身子，这是整个模型相对困难的一步。但其实把结构定义好了就会很简单。

你可以看一下这步变化的代码。

 复制代码

```
1  #[derive(Component)]
2  struct SnakeSegment;
3
4  #[derive(Resource, Default, Deref, DerefMut)]
5  struct SnakeSegments(Vec<Entity>);
6
7      // 插入蛇的结构，定义为资源
8      .insert_resource(SnakeSegments::default())
9
10 // 创建蛇，用SnakeSegments来维护蛇的结构
11 fn spawn_snake(mut commands: Commands, mut segments: ResMut<SnakeSegments>) {
12     *segments = SnakeSegments(vec![
13         commands
14             .spawn(SpriteBundle {
15                 sprite: Sprite {
16                     color: SNAKE_HEAD_COLOR,
17                     ..default()
18                 },
19                 ..default()
20             })
21             .insert(SnakeHead {
22                 direction: Direction::Up,
23             })
24             .insert(SnakeSegment)
25             .insert(Position { x: 3, y: 3 })
26             .insert(Size::square(0.8))
27             .id(),
28         spawn_segment(commands, Position { x: 3, y: 2 }),
29     ]);
30 }
31
32 // 创建蛇的一个segment，也就是一个小方块
33 fn spawn_segment(mut commands: Commands, position: Position) -> Entity {
```

```
34     commands
35         .spawn(SpriteBundle {
36             sprite: Sprite {
37                 color: SNAKE_SEGMENT_COLOR,
38                 ..default()
39             },
40             ..default()
41         })
42         .insert(SnakeSegment)
43         .insert(position)
44         .insert(Size::square(0.65))
45         .id()
46     }
```

这里，最关键的是定义了 SnakeSegment Component 和

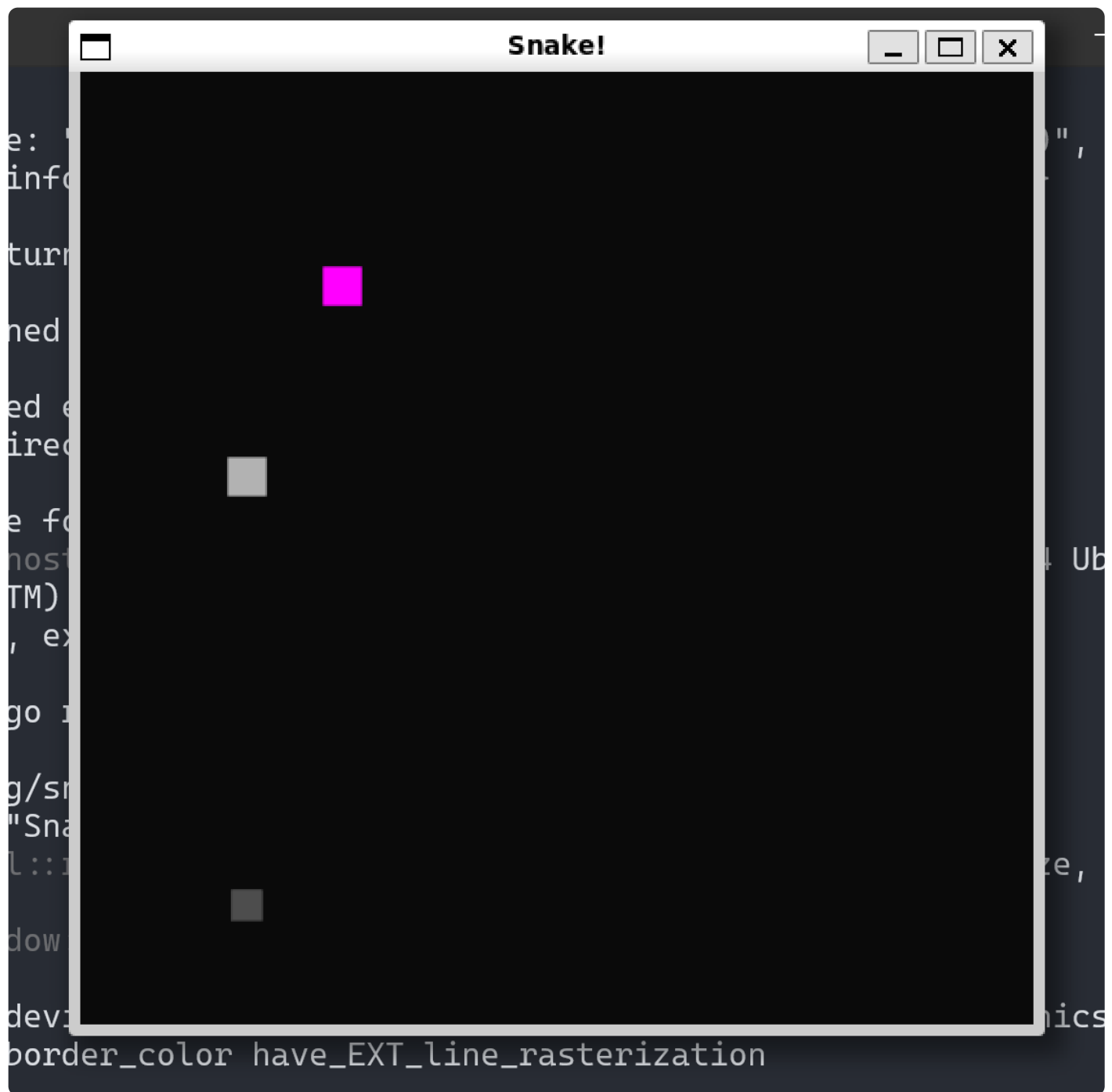
SnakeSegments(Vec<Entity>) 这个 Resource。我们把蛇的头和每一节身子小方块都视为一个 SnakeSegment，整条蛇由多个 SnakeSegment 组成，因此用

SnakeSegments(Vec<Entity>) 这个资源来维护这条蛇的结构。

SnakeSegments(Vec<Entity>) 里面需要存下每个 SnakeSegment 的 Entity id。

默认开始的时候，蛇有一节身子，位置在 (3, 2)。蛇的运动方向是向上的。蛇身小方块是 0.65 个网格单元大小。

你可以看一下这一步运行后的效果。



可以看到，这一节蛇身没有跟着头一起动。

第 13 步：让蛇身跟着蛇的头一起运动

让蛇身跟着蛇头一起动，模型上其实就是让蛇身的每一节跟着蛇头的移动一起变换坐标就行了。我们看一下这一步的代码变化。

```
1 fn snake_movement(
```

[复制代码](#)


```

2     time: Res<Time>,
3     mut timer: ResMut<BTimer>,
4     segments: ResMut<SnakeSegments>,
5     mut heads: Query<(Entity, &SnakeHead)>,
6     mut positions: Query<&mut Position>,
7 ) {
8     // 不到0.2s立即返回
9     if !timer.0.tick(time.delta()).finished() {
10         return;
11     }
12
13     if let Some((head_entity, head)) = heads.iter_mut().next() {
14         let segment_positions = segments
15             .iter()
16             .map(|e| *positions.get_mut(*e).unwrap())
17             .collect::<Vec<Position>>();
18         // 处理蛇的头的位移
19         let mut head_pos = positions.get_mut(head_entity).unwrap();
20         match &head.direction {
21             Direction::Left => {
22                 head_pos.x -= 1;
23             }
24             Direction::Right => {
25                 head_pos.x += 1;
26             }
27             Direction::Up => {
28                 head_pos.y += 1;
29             }
30             Direction::Down => {
31                 head_pos.y -= 1;
32             }
33         };
34         // 处理蛇身每一段的位置变化
35         segment_positions
36             .iter()
37             .zip(segments.iter().skip(1))
38             .for_each(|(pos, segment)| {
39                 *positions.get_mut(*segment).unwrap() = *pos;
40             });
41     }
42 }

```

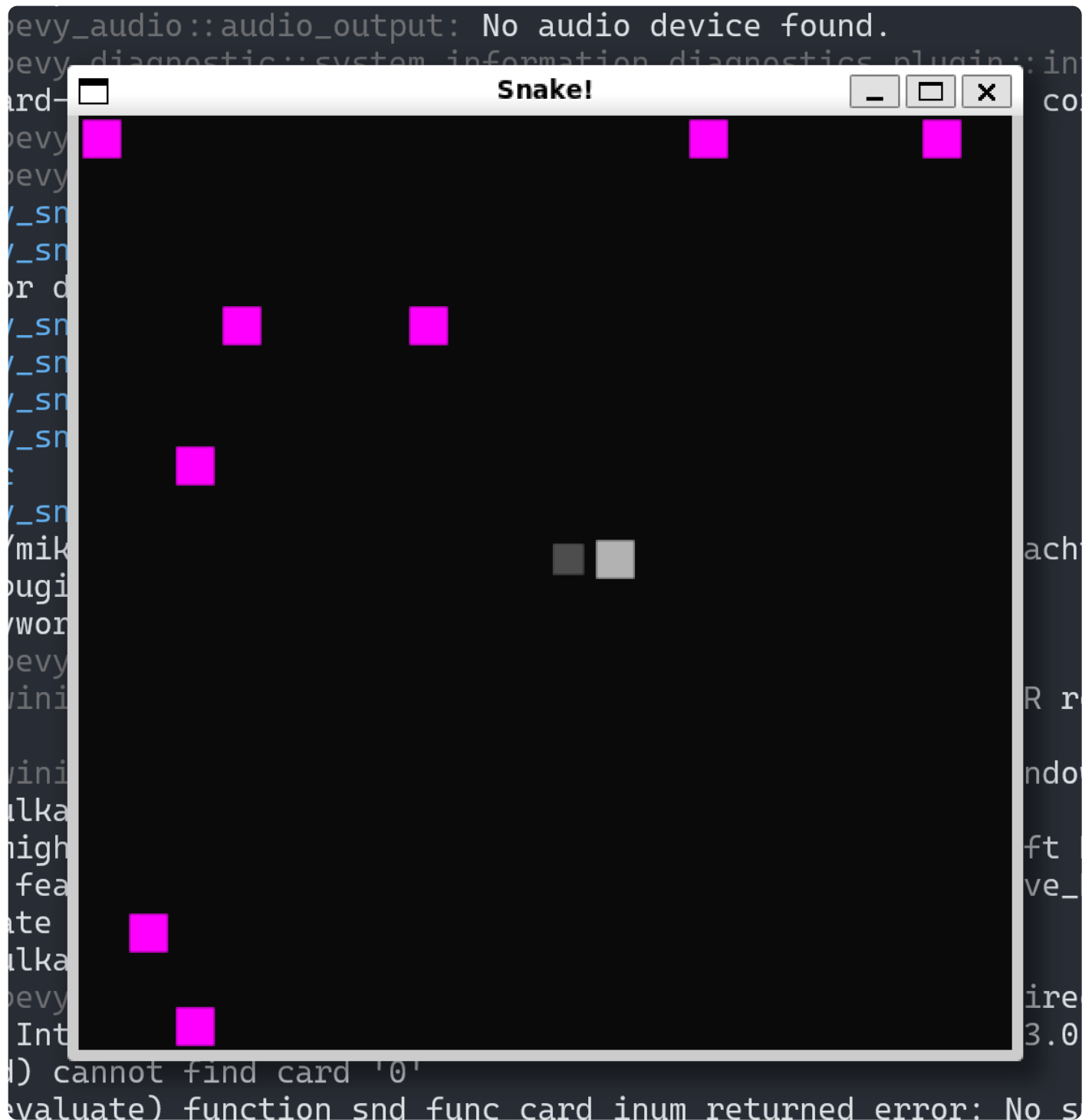
这个算法的精华在这一句：

```
2         .iter()  
3         .zip(segments.iter().skip(1))  
4         .for_each(|(pos, segment)| {  
5             *positions.get_mut(*segment).unwrap() = *pos;  
6         });
```

意思就是，当蛇动一步的时候，第一节蛇身的坐标值填充蛇头的坐标值，第二节蛇身的坐标值填充第一节蛇身的坐标值，以此类推，直到遍历完整个蛇身。

可以看到，Rust 可以把问题表达得相当精练。

你看一下这一步运行后的效果。



第 14 步：边吃边长大

下面就该处理吃食物并长大的效果了。吃食物的原理就是当蛇头占据了那个食物的位置时，就在系统中注销掉那个食物，然后在蛇身的尾巴位置处添加一个小方块。

你看一下这一步变化的代码。

```
1 #[derive(Event)]
2 struct GrowthEvent;
3
4 #[derive(Default, Resource)]
5 struct LastTailPosition(Option<Position>);
6
7 // 更新Update system集合
8 .add_systems(Update, (
9     snake_movement_input.before(snake_movement),
10    snake_movement,
11    snake_eating,
12    snake_growth,
13    size_scaling,
14    position_translation))
15
16 *last_tail_position = LastTailPosition(Some(*segment_positions.last().unwrap(
17
18 // 处理蛇吃食物的system
19 fn snake_eating(
20     mut commands: Commands,
21     mut growth_writer: EventWriter<GrowthEvent>,
22     food_positions: Query<(Entity, &Position), With<Food>>,
23     head_positions: Query<&Position, With<SnakeHead>>,
24 ) {
25     for head_pos in head_positions.iter() {
26         for (ent, food_pos) in food_positions.iter() {
27             // 通过遍历来判断有没有吃到一个食物
28             if food_pos == head_pos {
29                 commands.entity(ent).despawn();
30                 growth_writer.send(GrowthEvent);
31             }
32         }
33     }
34 }
35
36 // 处理蛇长大的system
37 fn snake_growth(
38     commands: Commands,
39     last_tail_position: Res<LastTailPosition>,
40     mut segments: ResMut<SnakeSegments>,
41     mut growth_reader: EventReader<GrowthEvent>,
42 ) {
43     // 通过事件机制来解耦蛇长大的逻辑
44     if growth_reader.read().next().is_some() {
45         segments.push(spawn_segment(commands, last_tail_position.0.unwrap()));
46     }
47 }
```

我们添加了 `LastTailPosition(Option<Position>)` 这个蛇尾的位置坐标作为资源来实时更新，好知道蛇长长的時候，应该在哪个位置添加 segment。然后新增了 `snake_eating` 和 `snake_growth` 两个 system。

我们新定义了 `GrowthEvent` 长大的事件。

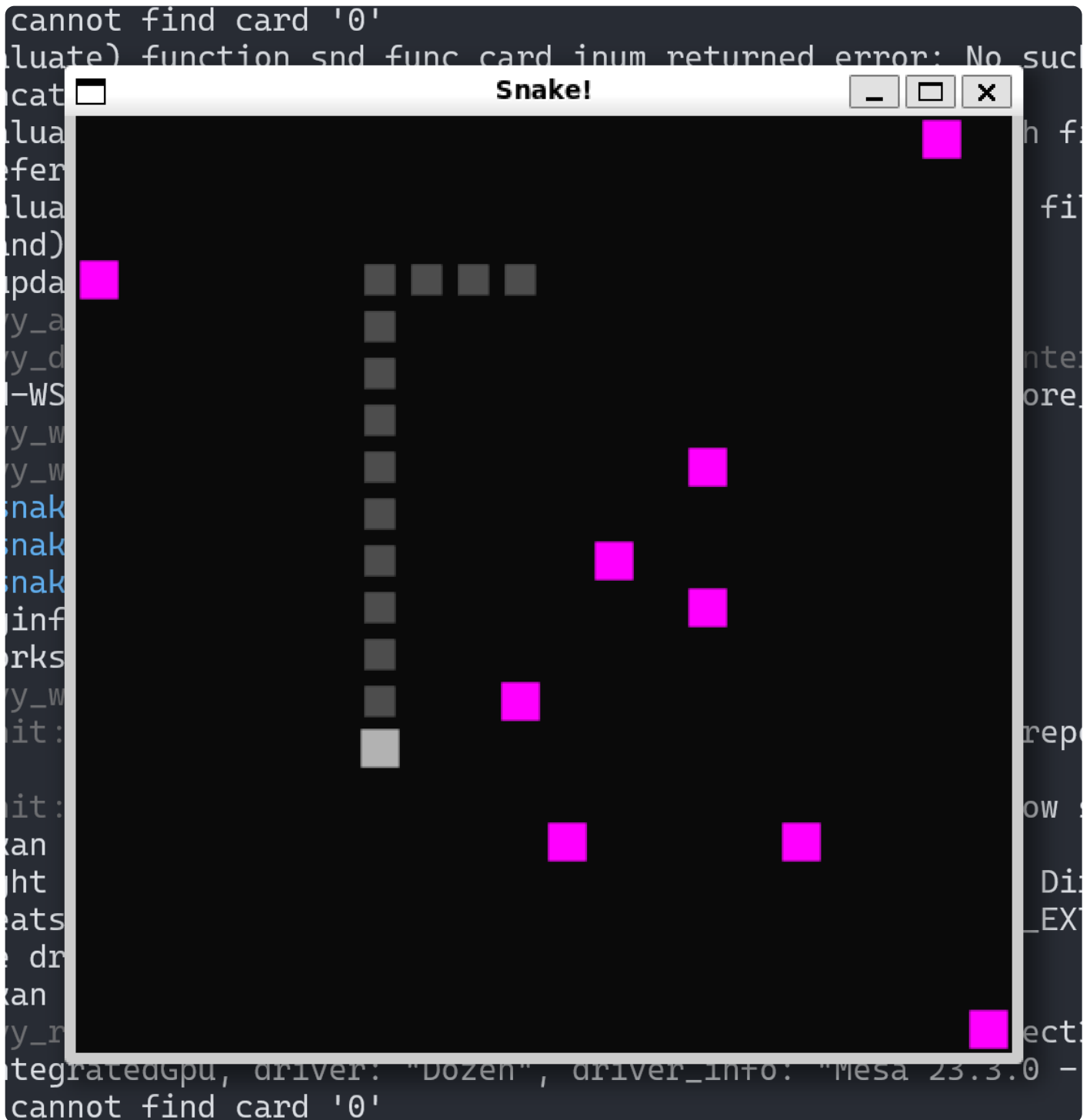
`snake_eating system` 处理吃食物的业务，就是当蛇头的位置与食物位置重合时，就调用 `commands.entity(ent).despawn()` 将食物给注销掉。然后用 `growth_writer.send(GrowthEvent)` 向系统总线发送一个“长大”的事件。

`snake_growth system` 处理蛇长大的业务，通过 `EventReader<GrowthEvent>` 定义的 `growth_reader`，读取系统中的长大事件，使用 `spawn_segment()` 和 `segments.push()` 把尾巴添加到蛇的全局维护资源中去。

`snake_eating` 和 `snake_growth` 在每一帧更新时都会执行。

可以看到，通过这样的事件总线，Bevy 系统把业务解耦得相当漂亮。每个 system 就专注于处理一件“小”事情就行了。这样对于构建复杂的游戏系统来说，大大减轻了我们的心智负担。

你可以看一下这一步执行后的效果。



第 15 步：撞墙和自身 Game Over

好了，我们的贪吃蛇的主体功能基本实现好了，下面需要加入撞墙和撞自身死的判断。你看一下这一步变化的代码。

 复制代码

```
1 #[derive(Event)]
2 struct GameOverEvent;
```

```

3      // 注册事件到world中
4      .add_event::<GameOverEvent>()
5      .add_systems(Update, (
6          snake_movement_input.before(snake_movement),
7          snake_movement,
8          game_over.after(snake_movement),
9          snake_eating,
10         snake_growth,
11         size_scaling,
12         position_translation))
13
14     // 判断撞墙的逻辑
15     if head_pos.x < 0
16         || head_pos.y < 0
17         || head_pos.x as u32 >= ARENA_WIDTH
18         || head_pos.y as u32 >= ARENA_HEIGHT
19     {
20         game_over_writer.send(GameOverEvent);
21     }
22     // 判断撞自己身子的逻辑
23     if segment_positions.contains(&head_pos) {
24         game_over_writer.send(GameOverEvent);
25     }
26     //
27
28     // game over 子系统
29     fn game_over(
30         mut commands: Commands,
31         mut reader: EventReader<GameOverEvent>,
32         segments_res: ResMut<SnakeSegments>,
33         food: Query<Entity, With<Food>>,
34         segments: Query<Entity, With<SnakeSegment>>,
35     ) {
36         if reader.read().next().is_some() {
37             for ent in food.iter().chain(segments.iter()) {
38                 commands.entity(ent).despawn();
39             }
40             spawn_snake(commands, segments_res);
41         }
42     }

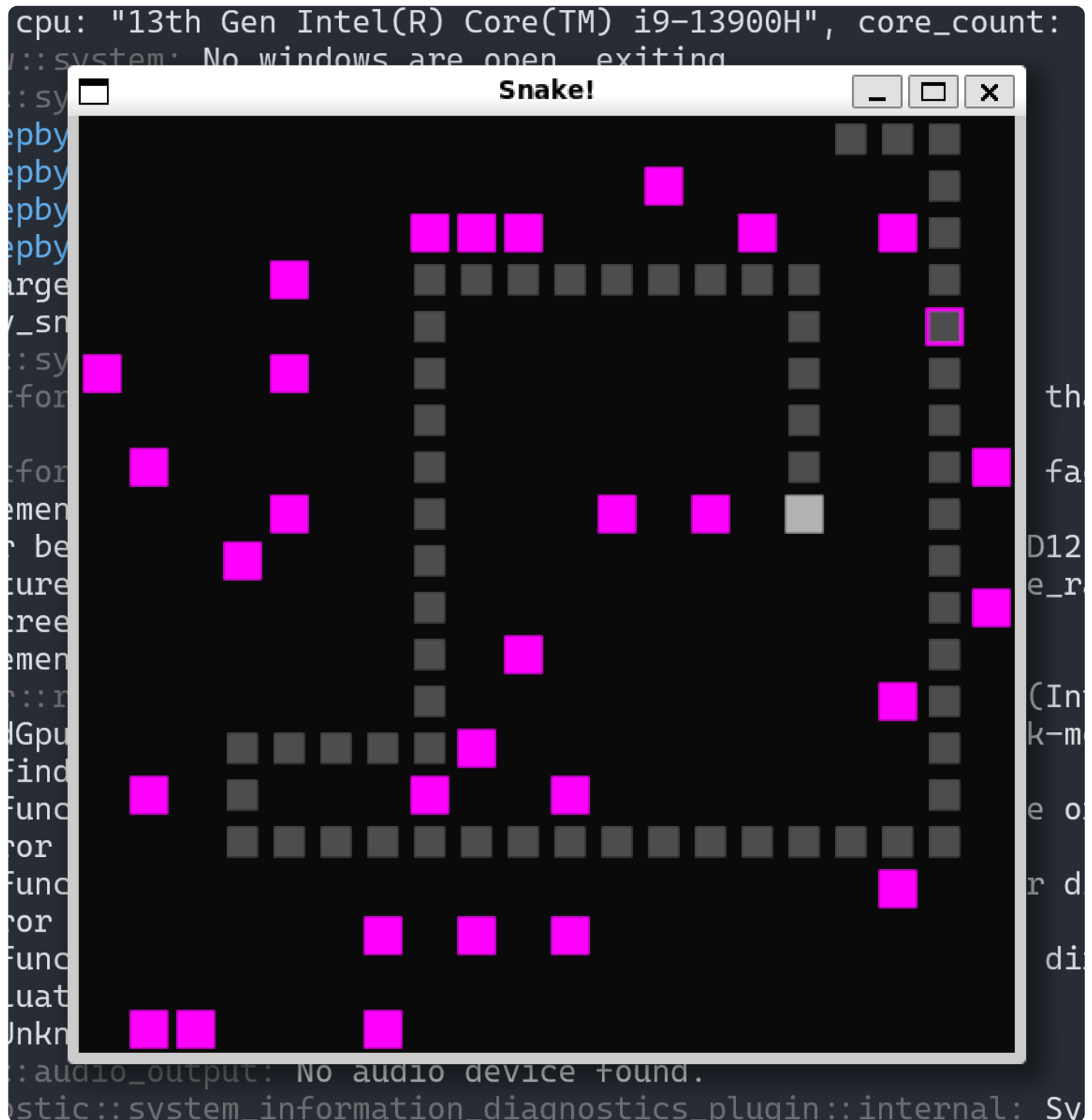
```

撞墙这个只需要判断有没有超出 grid 边界就可以了。撞自身判断用

`segment_positions.contains(&head_pos)` 看所有蛇身的 segment 的 position Vec 里有没有包含蛇头的位置。

我们添加了 `GameOverEvent` 事件和 `game_over` system，也是用的异步事件的方式。当收到 `GameOverEvent` 的时候，把所有的蛇的 entity 和食物的 entity 全部清理（despawn）掉。注意这里用了两个迭代器的 `.chain()` 方法，让清理工作表达得更紧凑，你可以体会一下。

清理完后，再重新创建蛇，游戏重新开始。到这一步，游戏已经基本能玩了，还写什么代码，先玩几把吧。



目前为止，整个代码不过 330 行左右。

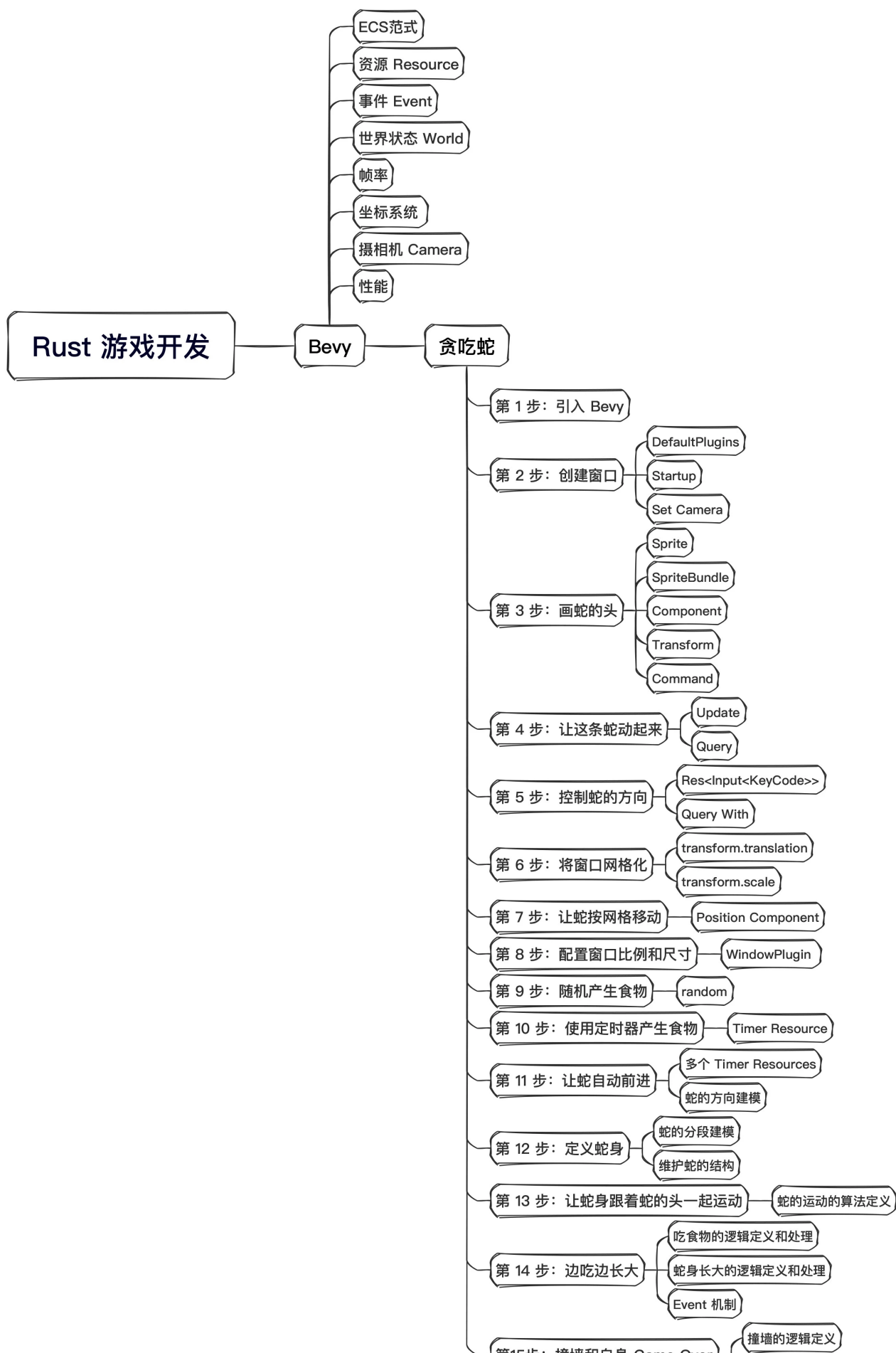
小结

这节课我们通过自己动手编写一个贪吃蛇小游戏，学习了 Rust 游戏开发引擎 Bevy 的基本使用方式。Bevy 游戏引擎充分利用 Rust 语言的无忧并发和强大的表达能力，让开发游戏变得跟游戏一样好玩。整个过程下来，心情愉快、舒畅。你可以跟着我一步一步敲代码，体会这种感觉。

Bevy 的核心是一套 ECS 系统，ECS 本质上来说是一套编程范式，不仅限于在游戏中使用，它也可以在其他业务系统中使用。你有必要多花点时间查阅相关资料去理解它。后面有机会我也会继续出相关的研究内容。

写 Bevy 代码的时候，我们要理解 Bevy 是一种 Runtime，我们写的代码实际会被这个 Runtime 托管运行。我们要做的就是按照 ECS 规范定义 Component、Resource、Event 等，实现 system 添加到这个 Runtime 中。底层那些脏活累活 Bevy 全帮我们做了，我们只需要专心抽象模型、定义结构、处理业务。

然后，通过这节课的内容我们可以体会到，写小游戏其实也是一种相当好的建模能力的训练，我们可以通过这种有趣的方法提升自己在这方面的能力。



本讲源代码： [🔗https://github.com/miketang84/jikeshijian/tree/master/27-bevy_snake](https://github.com/miketang84/jikeshijian/tree/master/27-bevy_snake)

必读的两个 Bevy 资料：

[🔗https://bevyengine.org/learn/book/introduction/](https://bevyengine.org/learn/book/introduction/)

[🔗https://bevy-cheatbook.github.io/introduction.html](https://bevy-cheatbook.github.io/introduction.html)

思考题

这节课的代码还有个问题，就是食物有可能在已经产生过的地方产生，也有可能在蛇身上产生，请问如何处理这个 Bug？欢迎你把你的处理思路和实现代码分享出来，我们一起探讨，如果你觉得这节课对你有帮助的话，也欢迎你把这节课的内容分享给其他朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (3)

最新 精选



unistart

2023-12-30 来自湖南

BUG: 食物可能在蛇身上产生

处理：在生成食物的时候，判断一下随机生成的Position值是否和当前蛇的某一个部分位置是重合的，如果是就直接return，不生成食物。

```
// 修改food_spawner
fn food_spawner(
    mut commands: Commands,
    time: Res<Time>,
    mut timer: ResMut<FoodSpawnTimer>,
```

```

segment_pos_set: Query<&Position, With<SnakeSegment>>
)
{
    // ...

    let rand_x: i32 = (random::<f32>() * ARENA_WIDTH as f32) as i32;
    let rand_y: i32 = (random::<f32>() * ARENA_HEIGHT as f32) as i32;

    for pos in segment_pos_set.iter() {
        if pos.x == rand_x && pos.y == rand_y {
            return;
        }
    }

    // ...
    .insert(Position {
        x: rand_x,
        y: rand_y,
    })
    // ...
}

```

BUG: 食物有可能在已经产生过的地方产生

个人感觉这个不太算bug吧，分两种情况看，一种是食物A之前被吃过，又在相同的位置生成了新的食物，这个应该是没问题的；另一种就是食物A之前没有被吃过，同时新生成的食物B的位置和食物A重合，这样确实貌似有问题。

但是我不知道Bevy是如何处理的，按我的理解来说食物B应该会盖掉食物A，即某一位置只会有一个最新的食物实体。还是说bevy中出现这种情况，同一位置会有多个实体。如果是同一位置有多个实体的话，那么再Query一下已经生成的Food的Position，然后和新生成的食物坐标比较一下是否有出现的重合就行了吧。

作者回复：非常棒的思考 🍌。要判断食物位置是否重合的话，需要用一個结构来维护食物的 Positions，将其设定为 Resource。在最新一篇总结更新中会讲到这个问题。



大白菜 🌱

2023-12-27 来自重庆

14步那里需要添加LastTailPosition资源， App需要添加 .insert_resource(LastTailPosition::default())

作者回复: 对的对的



十八哥

2023-12-26 来自天津

当年用vb.net用了1千行实现的。思路是按钮数组，一个二维数组。

作者回复: 不错的思路。

